

CSHRL Appendix

Supplementary Proofs: Subsumption, Stream Isomorphism, and Stochastic Extension

Ricardo Corral-Corral

February 2026

Abstract

This appendix provides supplementary machine-verified proofs for the main CSHRL paper. We establish three results: (1) the **Subsumption Theorem**—coinductive optimality implies classical argmax for any finite horizon; (2) the **Stream Isomorphism**—coinductive stream ordering is equivalent to pointwise dominance; and (3) the **Stochastic Subsumption Theorem**—extending argmax subsumption to stochastic MDPs via expected reward streams. We conclude with a conjecture on first-order stochastic dominance (FOSD) for risk-sensitive extensions. All proofs are literate Agda compiled with `--safe` and `--guardedness`.

1 Introduction

This document accompanies the main CSHRL paper with supplementary proofs. After establishing common definitions (§2), we present three machine-verified results and one conjecture:

1. **Subsumption Theorem** (§3): Coinductive optimality implies classical argmax optimality for any finite horizon N .
2. **Stream Isomorphism** (§4): $x \leq_s y \iff \forall n. x_n \leq_r y_n$ —the correspondence is bidirectional.
3. **Stochastic Subsumption** (§5): The subsumption theorem extends to stochastic MDPs via expected reward streams.
4. **FOSD Conjecture** (§6): A proposed extension to first-order stochastic dominance for risk-sensitive RL.

```
{-# OPTIONS --safe --guardedness #-}
```

```
module CSHRL-Appendix where
```

```
open import Data.List using (List; map; foldr; []; _::_)
open import Data.Nat using (ℕ; _⊔_)
open import Data.Product using (_×_; _,_; proj₁; proj₂)
open import Relation.Binary.PropositionalEquality using (_≡_; refl)
open import Codata.Guarded.Stream using (Stream; head; tail; tabulate)
```

2 Preliminaries

We begin by recalling the core definitions from the main development: reward streams, the coinductive stream ordering $\leq_s$, and the coinductive symmetric homomorphism record `CoindHomo`.

```

module PreservationEquivalence
  (State Action Reward : Set)
  (step      : State → Action → State × Reward)
  (≤r      : Reward → Reward → Set)
  (max       : Reward → Reward → Reward)
  (bottom    : Reward)
  (all-actions : List Action)
where

  -- Stream of rewards
  StreamR : Set
  StreamR = Stream Reward

  -- Optimal value at depth n
  max-list : List Reward → Reward
  max-list = foldr max bottom

  solve : State → ℕ → Reward
  solve s ℕ.zero = max-list (map (λ a → proj2 (step s a)) all-actions)
  solve s (ℕ.suc n) = max-list (map (λ a → solve (proj1 (step s a)) n) all-actions)

  value : State → StreamR
  value s = tabulate (solve s)

  action-value : State → Action → StreamR
  head (action-value s a) = proj2 (step s a)
  tail (action-value s a) = value (proj1 (step s a))

  -- Coinductive stream ordering
  record ≤s (x y : StreamR) : Set where
    coinductive
    field
      head≤ : head x ≤r head y
      tail≤ : tail x ≤s tail y

  open ≤s public

  -- The coinductive symmetric homomorphism
  record CoindHomo : Set1 where
    field
      ≤a : State → Action → Action → Set
      preserves : ∀ a b s → ≤a s a b →
        action-value s a ≤s action-value s b

```

3 Subsumption of Classical Argmax

The main paper establishes that CSHRL's coinductive optimality *subsumes* classical RL's argmax criterion. Here we provide the complete proof.

3.1 The Bridge: Pointwise Dominance

The key insight is that coinductive stream ordering ($\leq_s$) implies *pointwise* dominance at every index:

```

-- Extract the n-th element of a stream
iter-head : ℕ → StreamR → Reward
iter-head ℕ.zero s = head s
iter-head (ℕ.suc n) s = iter-head n (tail s)

```

```

-- Pointwise dominance: at every index
PointwiseDominance : StreamR → StreamR → Set
PointwiseDominance x y = ∀ n → iter-head n x ≤r iter-head n y

-- The bridge lemma
≤s-to-pointwise : ∀ {x y} → x ≤s y → PointwiseDominance x y
≤s-to-pointwise p ℕ.zero = head ≤ p
≤s-to-pointwise p (ℕ.suc n) = ≤s-to-pointwise (tail ≤ p) n

```

This lemma is the crucial connection: it converts the *infinite* coinductive structure into a property we can use for *finite* sums.

3.2 The Full Subsumption Module

The complete proof requires extending Core with reward arithmetic. The standalone implementation is in `src/appendix/Arithmetic.agda`. Here we show the module signature and key theorem:

```

module SubsumptionTheorem
  (State Action Reward : Set)
  (step    : State → Action → State × Reward)
  (≤r     : Reward → Reward → Set)
  (max     : Reward → Reward → Reward)
  (bottom  : Reward)
  (actions : List Action)
  -- Arithmetic extension
  (≤r     : Reward → Reward → Set)
  (≤r-refl : ∀ {r} → r ≤r r)
  (≤r-mono : ∀ {a b c d} → a ≤r b → c ≤r d → (a +r c) ≤r (b +r d))
  where

  open PreservationEquivalence State Action Reward step ≤r max bottom actions

  -- Partial sum definition
  partial-sum : ℕ → StreamR → Reward
  partial-sum ℕ.zero _ = bottom
  partial-sum (ℕ.suc n) s = head s +r partial-sum n (tail s)

  -- The subsumption theorem type
  SubsumesPartialSum : CoindHomo → Set
  SubsumesPartialSum homo = ∀ s a b N →
    CoindHomo.≤a homo s a b →
    partial-sum N (action-value s a) ≤r partial-sum N (action-value s b)

```

3.3 Proof Structure

The proof proceeds in three steps:

1. **Preservation:** Given $a \leq_a b$ in the ranking, `CoindHomo.preserves` yields $\text{action-value } s \ a \leq_s \text{action-value } s \ b$.
2. **Pointwise conversion:** Apply $\leq_s$ -to-pointwise to get $\forall n. r_n^a \leq_r r_n^b$.
3. **Induction on horizon:** Prove `partial-sum-mono` by induction on N :
 - Base ($N = 0$): Both sums equal bottom; use $\leq_r$ -refl.
 - Step ($N = k + 1$): The sum is $r_0 + \sum_{t=1}^k r_t$. Apply $\leq_r$ -mono to the head inequality and the inductive hypothesis on tails.

```

-- Helper for shifting pointwise to tails
pointwise-tail : ∀ {x y} → PointwiseDominance x y →
  PointwiseDominance (tail x) (tail y)
pointwise-tail pw n = pw (ℕ.suc n)

-- Partial sum monotonicity (the inductive core)
partial-sum-mono : ∀ N x y → PointwiseDominance x y →
  partial-sum N x ≤r partial-sum N y
partial-sum-mono ℕ.zero _ _ = ≤r-refl
partial-sum-mono (ℕ.suc n) x y pw =
  +-mono-≤ (pw ℕ.zero)
    (partial-sum-mono n (tail x) (tail y) (pointwise-tail pw))

-- The complete proof
subsumes-partial-sum : (h : CoindHomo) → SubsumesPartialSum h
subsumes-partial-sum h s a b N p =
  partial-sum-mono N (action-value s a) (action-value s b)
    (≤s-to-pointwise (CoindHomo.preserves h a b s p))

```

3.4 Corollary: Argmax Identification

An immediate corollary: if action b is ranked highest ($\forall a. a \leq_a b$), then b has the highest partial sum for *any* horizon:

```

argmax-subsumed : (h : CoindHomo) → ∀ s b →
  (∀ a → CoindHomo._≤a_ h s a b) →
  ∀ a N → partial-sum N (action-value s a) ≤r partial-sum N (action-value s b)
argmax-subsumed h s b b-top a N = subsumes-partial-sum h s a b N (b-top a)

```

This is the formal statement that **CSHRL rankings identify the classical argmax**—not by computing expected returns, but as a structural consequence of the coinductive homomorphism property.

4 Stream Isomorphism

The Subsumption Theorem uses one direction of a deeper correspondence: coinductive stream ordering implies pointwise dominance ($\leq_s$ -to-pointwise). We now prove the *converse*—pointwise dominance implies coinductive ordering—establishing a full isomorphism.

4.1 The Converse and the Isomorphism

```

-- Converse: pointwise dominance implies coinductive ordering
pointwise-to-≤s : ∀ {x y} → PointwiseDominance x y → x ≤s y
head≤ (pointwise-to-≤s pw) = pw ℕ.zero
tail≤ (pointwise-to-≤s pw) = pointwise-to-≤s (pointwise-tail pw)

_↔_ : Set → Set → Set
A ↔ B = (A → B) × (B → A)

-- The isomorphism: combining both directions
stream-iso : ∀ x y → (x ≤s y) ↔ (PointwiseDominance x y)
stream-iso x y = ≤s-to-pointwise , pointwise-to-≤s

```

This establishes:

$$x \leq_s y \iff \forall n. x_n \leq_r y_n$$

The proof of $\text{pointwise-to-}\leq_s$ is corecursive: to construct $x \leq_s y$ from pointwise dominance, we extract the head inequality from $\text{pw } 0$ and corecursively prove the tail inequality by shifting the pointwise witness via pointwise-tail .

4.2 Implications

The isomorphism has two consequences for the Finder algorithm, which defines rankings via trace comparison at finite depth:

1. **Finder** → **CoindHomo**: If traces dominate at all depths, streams dominate coinductively.
2. **CoindHomo** → **Finder**: If streams dominate coinductively, traces dominate at all depths.

Together, these show that the Finder’s ranking (at sufficient depth) *exactly* characterizes the coinductive property, not merely an approximation. The “symmetric” in CSHRL is thus formally justified: the correspondence between rankings and stream dominance is an isomorphism, not just a one-way homomorphism.

The standalone module `src/appendix/StreamIsomorphism.agda` provides the same proof in a reusable parameterized form.

5 Stochastic Subsumption Theorem

The deterministic subsumption theorem (§3) shows that coinductive optimality implies classical argmax for any finite horizon. A natural question: does this extend to *stochastic* MDPs?

5.1 The Key Insight: Linearity of Expectation

Classical RL maximizes *expected* cumulative reward:

$$\operatorname{argmax}_a \mathbb{E} \left[\sum_{t=0}^{N-1} r_t(a) \right]$$

By linearity of expectation:

$$\mathbb{E} \left[\sum_{t=0}^{N-1} r_t \right] = \sum_{t=0}^{N-1} \mathbb{E}[r_t]$$

This means comparing partial sums of *expected* rewards is equivalent to comparing *expected* partial sums—exactly what classical RL optimizes. The deterministic subsumption proof therefore applies directly to expected reward streams.

5.2 Stochastic Pointwise Dominance

For subsumption, we use **pointwise expected dominance** (stronger than lexicographic):

```
-- Pointwise expected stream dominance (for subsumption)
record _≤s-expected_ (x y : StreamR) : Set where
  coinductive
  field
    head≤ : head x ≤r head y
    tail≤ : tail x ≤s-expected tail y
```

Compare with the *lexicographic* version from the main stochastic theory:

```
-- Lexicographic: tail comparison is CONDITIONAL on head equality
record _≤s-lex_ (x y : StreamR) : Set where
  coinductive
  field
    head≤ : head x ≤r head y
    tail≤ : head x ≡ head y → tail x ≤s-lex tail y
```

The pointwise version requires tail dominance *unconditionally*, while lexicographic only requires it when heads are equal. Pointwise is strictly stronger: it implies lexicographic, but not vice versa.

5.3 The Stochastic Subsumption Theorem

The proof structure mirrors the deterministic case exactly:

```
-- Partial sum respects pointwise dominance
partial-sum-mono : ∀ N x y → PointwiseDominance x y →
  partial-sum N x ≤r partial-sum N y
partial-sum-mono ℕ.zero _ _ = ≤r-refl
partial-sum-mono (ℕ.suc n) x y pw =
  +-mono-≤ (pw ℕ.zero)
    (partial-sum-mono n (tail x) (tail y) (pointwise-tail pw))
```

Theorem 1 (Stochastic Subsumption). *If action rankings satisfy pointwise expected stream dominance, then for any finite horizon N , the ranked-higher action has higher expected cumulative reward:*

$$a \leq_{\text{rank}} b \wedge \mathbb{E}[v(a)] \leq_s^{\text{pw}} \mathbb{E}[v(b)] \implies \sum_{t=0}^{N-1} \mathbb{E}[r_t(a)] \leq \sum_{t=0}^{N-1} \mathbb{E}[r_t(b)]$$

The standalone implementation is in `src/appendix/StochasticSubsumption.agda`.

5.4 Two Orderings, Two Purposes

Ordering	Purpose	Role
Lexicographic ($\leq_s^{\text{lex}}$)	Trace-based learning, ranking semantics	Primary
Pointwise ($\leq_s^{\text{pw}}$)	Argmax subsumption, classical RL connection	Auxiliary

- **Lexicographic (primary):** The natural choice for CSHRL—aligned with finite trace learning and captures the “first difference decides” semantics. This is the ordering used in `StochasticCoindHomo`.
- **Pointwise (auxiliary):** A more restrictive ordering used *only* to establish the subsumption connection to classical RL. Many well-behaved MDPs satisfy pointwise when the ranking is sound.

The relationship: Pointwise $\Rightarrow$ Lexicographic, but not vice versa. Pointwise is more restrictive (requires dominance at *all* timesteps), while lexicographic is more permissive (only requires comparison when heads are equal). The lexicographic choice is intentional: it matches how finite trace comparison naturally extends to the infinite case.

6 Conjecture: First-Order Stochastic Dominance Isomorphism

6.1 Beyond Expected Values: Risk-Sensitive Orderings

The stochastic subsumption theorem (§5) compares *expected values*—collapsing distributions to their means. A richer question: can we compare *full distributions* over reward streams?

First-Order Stochastic Dominance (FOSD) is a classical ordering on probability distributions: distribution μ FOSD-dominates ν (written $\mu \geq_{\text{FOSD}} \nu$) iff for all thresholds r :

$$P_\mu(X \leq r) \leq P_\nu(X \leq r)$$

Equivalently, the CDF of μ lies everywhere below the CDF of ν . FOSD is strictly stronger than expected-value comparison: $\mu \geq_{\text{FOSD}} \nu \Rightarrow \mathbb{E}[\mu] \geq \mathbb{E}[\nu]$, but not vice versa.

6.2 The Conjecture

Let μ, ν be probability distributions over infinite reward streams. Define:

- **Coinductive FOSD:** $\mu \leq_s \nu$ iff the marginal distribution of heads satisfies FOSD, and the conditional tail distributions satisfy $\mu_{\text{tail}} \leq_s \nu_{\text{tail}}$ (corecursively).
- **Pointwise FOSD:** $\mu \leq_{\text{pw}} \nu$ iff for all timesteps n and thresholds r , $P_\mu(X_n \leq r) \geq P_\nu(X_n \leq r)$.

Conjecture (Stochastic Isomorphism): $\mu \leq_s \nu \iff \mu \leq_{\text{pw}} \nu$

If true, this would extend CSHRL to stochastic environments with a *risk-sensitive* interpretation: a policy that stochastically dominates is preferred by *all* risk-averse decision makers, not just those maximizing expected return.

6.3 Current Infrastructure and Remaining Gap

The Giry monad foundation is now in place (`CSHRL.Probability.Finite`): finite distributions as weighted lists, with monadic operations (`pure`, `>>=`, `fmap`) and expected value computation. The stochastic extension (`CSHRL.Core.Stochastic`) uses this to define expected reward streams and lexicographic comparison.

However, proving the FOSD isomorphism conjecture requires additional infrastructure not yet implemented:

- **CDF computation:** Cumulative distribution functions for `Dist Reward`, requiring decidable ordering on rewards.
- **FOSD comparison:** $d_1 \leq_{\text{FOSD}} d_2$ iff $\forall r. \text{cdf}(d_1, r) \geq \text{cdf}(d_2, r)$.
- **Conditional tail distributions:** Preserving correlation structure through coinductive unfolding.

The current expected-value approach (§5) is sufficient for classical RL subsumption. The FOSD extension would enable *risk-sensitive* comparisons—a distinct research direction beyond expected-value optimization.